Data Structures and Systems Design

Keyur Sherke | Avaloka Internship Assignment | Python

Code and tests: github.com/KEYUR141/lru-cache-event-scheduler

Problem 1 LRU Cache

I need fast lookup as well as fast changes to usage order. I use a dictionary mapping each key to its node in a doubly linked list. The dictionary finds a node in average O(1) time, and its previous and next pointers let me detach and reposition it in O(1) time without searching the list.

The list runs from least recent to most recent. Two dummy sentinel nodes mark its ends: `head.next` is the least recent real entry and `tail.prev` is the most recent. Sentinels avoid separate insertion and removal cases for an empty list or a single entry.

Implementation and list helpers

```
"""LRU cache: head.next is least recent; tail.prev is most recent.

get and put take average O(1) time. This class is not thread-safe.
"""

from collections.abc import Hashable


class _Node:
    def __init__(self, key=None, value=None):
        self.key = key
        self.value = value
        self.next = None
        self.prev = None


class LRUCache:
    """Cache integer values under hashable keys; a miss returns -1."""

    def __init__(self, capacity: int):
        if type(capacity) is not int:
            raise TypeError("Capacity must be an integer")
        if capacity < 0:
            raise ValueError("Capacity cannot be negative")

        self.capacity = capacity
        self.nodes = {}

        self.head = _Node()
        self.tail = _Node()

        self.head.next = self.tail
        self.tail.prev = self.head

    def _remove(self, node: _Node) -> None:
        previous_node = node.prev
        next_node = node.next

        previous_node.next = next_node
        next_node.prev = previous_node

        node.prev = None
        node.next = None

    def _add_to_most_recent(self, node: _Node) -> None:
        previous_node = self.tail.prev

        previous_node.next = node
        node.prev = previous_node
        node.next = self.tail
        self.tail.prev = node
```

Problem 1 Cache Operations

For `get`, I return -1 for a missing key. Otherwise, I move its node to the most recent end and return the value. For `put`, I first handle existing keys by updating and moving their nodes. For a new key at capacity, I remove the least recent node from both structures, then insert the new entry at the most recent end. Each node stores its key so I can delete the dictionary entry during eviction without searching.

```
    def get(self, key: Hashable) -> int:
        if key not in self.nodes:
            return -1

        node = self.nodes[key]

        self._remove(node)
        self._add_to_most_recent(node)

        return node.value

    def put(self, key: Hashable, value: int) -> None:
        if key in self.nodes:
            node = self.nodes[key]
            node.value = value

            self._remove(node)
            self._add_to_most_recent(node)
            return

        if self.capacity == 0:
            return

        if len(self.nodes) == self.capacity:
            least_recent = self.head.next

            self._remove(least_recent)
            del self.nodes[least_recent.key]

        node = _Node(key, value)
        self.nodes[key] = node
        self._add_to_most_recent(node)

    def display(self) -> None:
        current = self.head.next
        entries = []

        while current is not self.tail:
            entries.append(str(current.key))
            current = current.next

        if entries:
            print("  <------>  ".join(entries))
        else:
            print("Cache is empty")
```

Correctness and example

Every real node is represented exactly once in the dictionary and list. Lookup, update, insertion, and eviction preserve that invariant and keep the list in usage order. Thus the entry evicted from the head is always the least recently used.

With capacity 2, `put(A, 10)`, `put(B, 20)`, `get(A)`, `put(C, 30)` changes the order from A-B to B-A to A-C. B is evicted because accessing A refreshed its recency.

Capacity zero stores nothing. Negative capacity is invalid. The optional `display` helper traverses the list for learning and debugging, but neither cache operation calls it.

Keys are hashable and integer values are intended. Missing keys return -1; storing -1 therefore has the same return value as a miss. Capacity must be a nonnegative integer; booleans and other types are rejected.

Problem 2 Event Scheduler

I interpret each meeting as a half-open interval [start, end). Equality between one end and another start is allowed. I validate `start < end`, then sort a copy by start time so the original input remains unchanged.

For `can_attend_all`, I compare each start with the previous meeting's end. If the start is smaller, those meetings overlap and I return False. If every adjacent pair passes, their chronological chain has no overlap, so I return True. Empty and single-event inputs also return True.

For `min_rooms_required`, I keep a min-heap of end times for active meetings. Before adding a meeting, I remove all end times less than or equal to its start; those meetings have released their rooms. I push the new end time and record the largest heap size seen. A heap exposes its smallest value at index zero, so I can repeatedly find the next meeting to finish without scanning all active meetings.

After each insertion, the heap contains exactly the meetings active at that start time. The peak heap size is necessary because simultaneous meetings need different rooms. It is sufficient because a room can be reused whenever its previous meeting finishes, and occupancy rises only when a meeting starts. Therefore the peak is the minimum number of rooms.

For [(9, 12), (10, 11), (11, 13)], the heaps after insertion are [12], [11, 12], and [12, 13]. The end time 11 is removed before the last insertion, so two rooms suffice. I return the maximum, not the final heap size, because the busiest moment may occur earlier.

Implementation

```
"""Meeting attendance and room counts for half-open intervals [start, end)."""

import heapq


def _sorted_events(events):
    """Validate comparable time pairs and sort a copy by start time."""
    copied_events = list(events)
    for start, end in copied_events:
        if start >= end:
            raise ValueError("Each event must have start < end")
    return sorted(copied_events)


def can_attend_all(events):
    """Return True if one person can attend every event without overlap."""
    sorted_events = _sorted_events(events)
    for index in range(1, len(sorted_events)):
        previous_end = sorted_events[index - 1][1]
        current_start = sorted_events[index][0]
        if current_start < previous_end:
            return False
    return True


def min_rooms_required(events):
    """Return peak simultaneous meetings, allowing adjacent room reuse."""
    sorted_events = _sorted_events(events)
    active_end_times = []
    max_rooms = 0

    for start, end in sorted_events:
        # End times equal to start also release their rooms.
        while active_end_times and active_end_times[0] <= start:
            heapq.heappop(active_end_times)
        heapq.heappush(active_end_times, end)
        max_rooms = max(max_rooms, len(active_end_times))

    return max_rooms
```

Inputs use comparable, finite time values in a common unit. Duplicate meetings are distinct events. Empty input needs zero rooms. Neither function changes the input list.

Final Discussion and Analysis

1 Time and space complexity

Let n be the number of events, c the configured cache capacity, and k the number of entries currently cached. Cache keys are assumed to have constant-cost hashing and equality, as with the small integer and string keys used here. Space below is auxiliary space unless stated otherwise.

Function	Time	Space	Reason
_Node.__init__	O(1)	O(1) per node	Initializes four fields
LRUCache.__init__	O(1)	O(1) initially	Empty map and two sentinels; no preallocation
_remove	O(1)	O(1)	Fixed number of pointer changes
_add_to_most_recent	O(1)	O(1)	Fixed number of pointer changes
get	Average O(1)	O(1)	Dictionary lookup and list movement
put	Average amortized O(1)	O(1) per operation, excluding dictionary resize	At most one eviction and one insertion
display	O(k)	O(k)	Collects and joins printed key strings
_sorted_events	O(n log n) worst case	O(n)	Copies, validates, and sorts events
can_attend_all	O(n log n) worst case	O(n)	Sorting plus a linear adjacent scan
min_rooms_required	O(n log n) worst case	O(n)	Sorting and heap operations

The cache as a whole uses O(k + 1) storage, bounded by O(c + 1), for the dictionary, data nodes, and two sentinels. Cache complexities assume bounded-size keys and values; display also depends on the total size of the strings printed. Dictionary resizing can make an individual insertion O(k), and pathological hash collisions can make dictionary operations O(k). The required O(1) cache performance is the conventional average/amortized guarantee of hash-map implementations.

In the scheduler, validation takes O(n). Python sorting and the copied lists require O(n) storage. The heap itself holds at most r end times, where r is peak occupancy and r <= n. Each meeting is pushed once and popped at most once across the entire loop, so the nested while loop does not make the algorithm quadratic. Total worst-case time is O(n log n) and auxiliary space is O(n).

2 LRU data structure trade-offs

A dictionary alone provides fast lookup but a plain key-value lookup interface does not express how to reposition and evict entries by recency. A doubly linked list alone maintains order but needs O(k) lookup. Combining them provides both operations efficiently, at the cost of a dictionary entry, a node, and two links per cached item, plus the responsibility of keeping both structures synchronized.

A Python list requires O(k) shifting when removing an arbitrary entry. A singly linked list cannot unlink a known node in O(1) without also knowing its predecessor. A timestamp heap makes recency updates or eviction O(log k), and lazy updates can accumulate stale entries. `collections.OrderedDict` offers a shorter practical implementation through `move_to_end` and `popitem(last=False)`; I implemented the linked list explicitly to demonstrate the required mechanics. Python's regular dictionaries preserve insertion order, but that alone does not automatically maintain access order.

Room Assignment and Thread Safety

3 Assigning specific rooms

I would sort `(start, end, original_index)` records and replace the end-time heap with a busy-room heap containing `(end, room_id)`. I would also maintain a min-heap of available room IDs and a counter for new IDs.

Before each meeting, I would pop every finished meeting from the busy heap and return its ID to the available heap. I would reuse the smallest available ID, or allocate a new ID if none is free, then push `(end, room_id)` into the busy heap. I would save the assignment at `assignments[original_index]` so unsorted and duplicate input events retain their individual identities. Integer IDs can be displayed as Room 1, Room 2, and so on, or mapped to Room A, Room B, and further labels.

For [(9, 12), (10, 11), (11, 13)], the assignments can be [Room A, Room B, Room B]. A new room is allocated only when all existing rooms are occupied, so this uses the minimum number of rooms. Time remains O(n log n), and auxiliary space remains O(n). This is a proposed extension, not an extra implemented API.

4 Making the cache thread-safe

I would create one `threading.Lock` for each cache instance and hold it for the entire body of every public `get` and `put`. A successful `get` modifies the linked list, so reads also require this lock. Lookup, movement, eviction, and dictionary insertion must form one atomic operation with respect to other callers; locking individual dictionary accesses is insufficient.

The private list helpers would assume their caller holds the lock. They would not acquire it again, avoiding nested acquisition of a non-reentrant lock. For `display`, I would copy the keys while holding the same lock, then release it before formatting and printing the snapshot. Any added public inspection method would also use that lock. Callers must not mutate internal nodes or the dictionary directly.

Python's GIL does not guarantee atomicity of these multi-step operations and is not a replacement for the cache lock. A single lock is simple and preserves a global LRU order, but serializes callers; waiting time under contention is not covered by the O(1) algorithmic bound. Sharding can improve throughput, but changes exact global LRU behavior unless extra coordination is added. This design addresses threads within one process; separate processes would need shared storage and process-safe coordination. The submitted cache itself is single-threaded.

Verification

Run `python -m unittest discover -s tests -v` from the repository root. There are 13 tests, including 1,000 deterministic randomized cache operations checked against OrderedDict and 200 schedules checked against independent overlap and occupancy calculations. Tests also verify linked-list consistency and the boundary behaviors documented in the README.